

Objetivo

Inspeccionar la relación entre Deployment, ReplicaSet, Pod y Service. Diagnosticar un fallo preexistente sin conocer previamente su causa. Utilizar kubectl get, describe, labels, selectors y endpoints como evidencia. Corregir únicamente el recurso responsable y validar la recuperación del servicio.

Requisitos previos

- Haber completado la Práctica 1.
- Conservar node-web-deployment y node-web-service.
- Tener Minikube y kubectl operativos.
- Disponer del archivo lab1-1-scenario.yaml proporcionado con esta práctica.

Duración estimada

20 minutos

Introducción

En este reto recibirás una aplicación que funcionaba correctamente al finalizar la Práctica 1, pero cuyo acceso mediante Service ha dejado de funcionar. No conocerás de antemano la causa. Tu objetivo será observar el estado de los recursos, recopilar evidencia, identificar la relación rota y aplicar la corrección mínima necesaria sin reiniciar innecesariamente la aplicación.

Instrucciones



Escenario

La aplicación Node.js desplegada en la Práctica 1 continúa ejecutándose, pero un cambio reciente ha provocado que el acceso a través de `node-web-service` deje de funcionar.

No se proporciona la causa del problema.

Tu objetivo es responder:

¿Qué recurso está fallando?
¿Qué evidencia demuestra la causa?
¿Cuál es la corrección mínima necesaria?
¿Cómo compruebas que el servicio volvió a funcionar?

 Copiar

Distribución sugerida

Preparación	2 min
Reto 1	6 min
Reto 2	7 min
Reto 3	5 min
Total	20 min

 Copiar

Preparación del escenario

Paso 1. Confirmar los recursos de la Práctica 1

- **Paso 1.** Verifica que el Deployment y el Service originales continúan disponibles antes de introducir el incidente.

```
kubectl get deployment node-web-deployment  
kubectl get service node-web-service  
kubectl get pods -l app=node-web
```

 Copiar

Resultado esperado:

El Deployment debe estar disponible y al menos un Pod debe encontrarse en estado `Running`.

Paso 2. Aplicar el escenario defectuoso

- **Paso 2.** Ejecuta el siguiente comando para descargar el archivo del reto.

```
curl -L -o k8s/lab1-1-scenario.yaml  
https://raw.githubusercontent.com/Netec-  
Mx/CKA/refs/heads/main/labs/lab2/lab2-  
scenario.yaml
```

 Copiar

- **Paso 3.** Desde el directorio donde guardaste `lab2-scenario.yaml`, aplica el archivo sin abrirlo ni inspeccionarlo todavía.

```
kubectl apply -f lab1-1-scenario.yaml
```

 Copiar

IMPORTANTE: El archivo contiene el cambio que originó el incidente. No utilices `cat`, VS Code ni otro editor para inspeccionarlo antes de completar el diagnóstico.

- **Paso 4.** Espera unos segundos para que Kubernetes actualice el estado relacionado con el Service.

```
sleep 5
```

 Copiar

Reto 1. Determinar qué sigue funcionando

Tiempo sugerido: 6 minutos

Antes de buscar la causa, separa los componentes sanos de los componentes afectados.

Reto 1.1. Revisar el workload

- **Paso 5.** Inspecciona el Deployment y los Pods sin modificar ningún recurso.

```
kubectl get deployment node-web-deployment  
kubectl get pods -l app=node-web -o wide
```

 Copiar

Debes responder:

```
¿El Deployment mantiene las réplicas esperadas?  
¿Los Pods están Running?  
¿Los Pods tienen IP?  
¿La aplicación parece haber fallado realmente?
```

[Copiar](#)

Reto 1.2. Probar la aplicación directamente dentro del Pod

- **Paso 6.** Comprueba el endpoint `/health` directamente dentro de uno de los Pods.

```
kubectl exec deployment/node-web-deployment -- \\\n  wget -qO- http://localhost:3000/health
```

[Copiar](#)

Resultado esperado aproximado:

```
{"status":"ok","pod":"node-web-deployment-<hash>-<id>"}
```

[Copiar](#)

Pregunta de diagnóstico

Si el Pod responde directamente pero el acceso mediante Service falla:

```
¿Deberías investigar primero la aplicación o la  
relación entre el Service y sus backends?
```

[Copiar](#)

Reto 1.3. Revisar el Service

- **Paso 7.** Comprueba que el Service todavía existe y conserva una ClusterIP.

```
kubectl get service node-web-service
```

[Copiar](#)

- **Paso 8.** Intenta acceder a la aplicación mediante el nombre del Service desde el propio workload.

```
kubectl exec deployment/node-web-deployment -- \
  wget -qO- \
  --timeout=3 \
  http://node-web-service/health
```

 Copiar

Registra si la conexión responde o falla.

Reto finalizado

Resultado esperado: Estado del Deployment, ReplicaSet, Pods y Service analizado correctamente, diferenciando los componentes saludables del recurso afectado.



Reto 2. Encontrar la causa raíz

Tiempo sugerido: 7 minutos

Ahora debes encontrar por qué un Service existente no puede llegar a una aplicación que continúa saludable.

Reto 2.1. Revisar backends asociados

- **Paso 9.** Comprueba si `node-web-service` tiene destinos disponibles.

```
kubectl get endpoints node-web-service
```

 Copiar

- **Paso 10.** Revisa también el recurso moderno EndpointSlice asociado con el Service.

```
kubectl get endpointslices \
  -l kubernetes.io/service-name=node-web-service
```

 Copiar

Evidencia requerida

Registra:

¿Existen direcciones de backend?
¿Aparece alguna IP de Pod?
¿El Service tiene un destino real al cual enviar tráfico?

 Copiar

Reto 2.2. Inspeccionar configuración efectiva

- **Paso 11.** Describe el Service y localiza Selector y Endpoints .

```
kubectl describe service node-web-service
```

 Copiar

- **Paso 12.** Muestra los labels reales de los Pods administrados por el Deployment.

```
kubectl get pods \  
-l app=node-web \  
--show-labels
```

 Copiar

- **Paso 13.** Consulta también el selector configurado directamente en el Service.

```
kubectl get service node-web-service \  
-o jsonpath='{.spec.selector}{"\n"}'
```

 Copiar

Diagnóstico

Compara:

Selector del Service
vs.
Labels de los Pods

 Copiar

No avances hasta poder explicar qué relación está rota.

Reto 2.3. Confirmar que el Deployment no debe modificarse

- **Paso 14.** Revisa el selector del Deployment para comprobar que sus Pods están correctamente administrados.

```
kubectl get deployment node-web-deployment \
  -o jsonpath='{.spec.selector.matchLabels}'
{"\n"}
```

 Copiar

Criterio de diagnóstico

Debes poder concluir cuál de estas afirmaciones corresponde al escenario:

- A. El Deployment no crea Pods.
- B. La aplicación falla dentro del contenedor.
- C. El Service existe, pero no selecciona los Pods correctos.
- D. Kubernetes DNS no puede resolver node-web-service.

 Copiar

Reto finalizado

Resultado esperado: Causa raíz del fallo identificada mediante `kubectl`, labels, selectors, endpoints y pruebas de conectividad, sin modificar recursos innecesarios.

Reto 3. Aplicar la corrección mínima

Tiempo sugerido: 5 minutos

Ya identificaste la causa raíz. Corrige únicamente el recurso responsable.

Restricciones

- [] No elimines node-web-deployment.
- [] No reinicies los Pods.
- [] No cambies los labels de los Pods.
- [] No recrees node-web-service.
- [] Conserva la ClusterIP actual.

 Copiar

Reto 3.1. Registrar la ClusterIP actual

- **Paso 15.** Guarda la ClusterIP antes de corregir el incidente.

```
SERVICE_IP=$(kubectl get service node-web-  
service \  
  -o jsonpath='{.spec.clusterIP}')
```

echo "ClusterIP antes de la corrección:
\$SERVICE_IP"

 Copiar

Reto 3.2. Corregir el recurso

- **Paso 16.** Utiliza `kubectl patch`, `kubectl edit` o un manifiesto corregido para restaurar la relación adecuada entre el Service y los Pods.

Pistas permitidas

```
kubectl patch service  
kubectl edit service  
kubectl get pods --show-labels  
kubectl get service -o yaml
```

 Copiar

No se proporciona el comando exacto porque forma parte del reto.

Reto 3.3. Validar recuperación

- **Paso 17.** Comprueba que Kubernetes vuelva a publicar destinos para el Service.

```
kubectl get endpoints node-web-service
```

 Copiar

- **Paso 18.** Verifica EndpointSlice.

```
kubectl get endpointslices \
-l kubernetes.io/service-name=node-web-service
```

 Copiar

- **Paso 19.** Comprueba nuevamente `/health` utilizando el nombre del Service.

```
kubectl exec deployment/node-web-deployment -- \
wget -qO- http://node-web-service/health
```

 Copiar

- **Paso 20.** Confirma que la ClusterIP no cambió durante la reparación.

```
NEW_SERVICE_IP=$(kubectl get service node-web-
service \
-o jsonpath='{.spec.clusterIP}')

echo "Antes:  $SERVICE_IP"
echo "Después: $NEW_SERVICE_IP"
```

 Copiar

Validación final

Ejecuta:

```
echo "=== Validacion final Lab 2 ==="

echo ""
echo "--- Deployment ---"
kubectl get deployment node-web-deployment

echo ""
echo "--- Pods ---"
kubectl get pods -l app=node-web -o wide

echo ""
echo "--- Service ---"
kubectl get service node-web-service

echo ""
echo "--- Selector ---"
kubectl get service node-web-service \
  -o jsonpath='{.spec.selector}{"\n"}'

echo ""
echo "--- Endpoints ---"
kubectl get endpoints node-web-service

echo ""
echo "--- Health mediante Service ---"
kubectl exec deployment/node-web-deployment -- \
  wget -qO- http://node-web-service/health

echo ""
echo "=== Fin de validacion ==="
```

 Copiar

El reto queda completado si:

```
[✓] node-web-deployment continúa disponible.
[✓] Los Pods permanecen Running.
[✓] node-web-service conserva su ClusterIP.
[✓] El Service vuelve a tener endpoints.
[✓] /health responde mediante node-web-service.
[✓] No fue necesario reiniciar ni recrear el
Deployment.
```

 Copiar



Cierre del reto

La evidencia debe llevarte a reconstruir el flujo:

[Copiar](#)

La clave del troubleshooting no consiste en modificar recursos hasta que algo funcione, sino en:

1. comprobar qué componente sigue sano;
2. localizar dónde se rompe la relación;
3. obtener evidencia;
4. corregir únicamente la causa;
5. validar funcionalmente la recuperación.

[Copiar](#)

Resolución de problemas

Problema 1. El archivo de escenario no se encuentra

Comprueba el directorio actual:

```
pwd
ls -la
```

[Copiar](#)

`lab2-scenario.yaml` debe encontrarse en el directorio desde el que ejecutas `kubectl apply`.

Problema 2. El Pod tampoco responde directamente

Si esta prueba falla:

```
kubectl exec deployment/node-web-deployment -- \
  wget -q0- http://localhost:3000/health
```

[Copiar](#)

el escenario ya no coincide con el reto esperado.

Antes de continuar, revisa:

```
kubectl get pods -l app=node-web
kubectl logs deployment/node-web-deployment
```

[Copiar](#)

La aplicación de la Práctica 1 debe estar saludable antes de diagnosticar el Service.

Problema 3. Los endpoints no reaparecen después de corregir el Service

Compara nuevamente:

```
kubectl get service node-web-service \
  -o jsonpath='{.spec.selector}{"\n"}'

kubectl get pods \
  -l app=node-web \
  --show-labels
```

[Copiar](#)

El selector del Service debe coincidir exactamente con los labels requeridos en los Pods.

¡Felicidades! Has completado el Laboratorio.

Resultado final

Al finalizar el reto habrás diagnosticado un fallo realista de conectividad entre un Service y sus Pods, identificado la causa mediante kubectl y restaurado el acceso sin recrear el Deployment.

Notas y consideraciones

- El escenario contiene un error intencional que no debe revisarse antes de iniciar el diagnóstico.
- No elimines ni recrees el Deployment node-web-deployment.
- No modifiques los labels de los Pods para resolver el incidente.
- Esta práctica complementaria no incluye prompts de apoyo con IA.

URL de referencia

- [Debug Services](#)
- [Labels and Selectors](#)
- [Service](#)